

Fundamentals of Deep Learning and Programming

Lecture 2: PyTorch Fundamentals

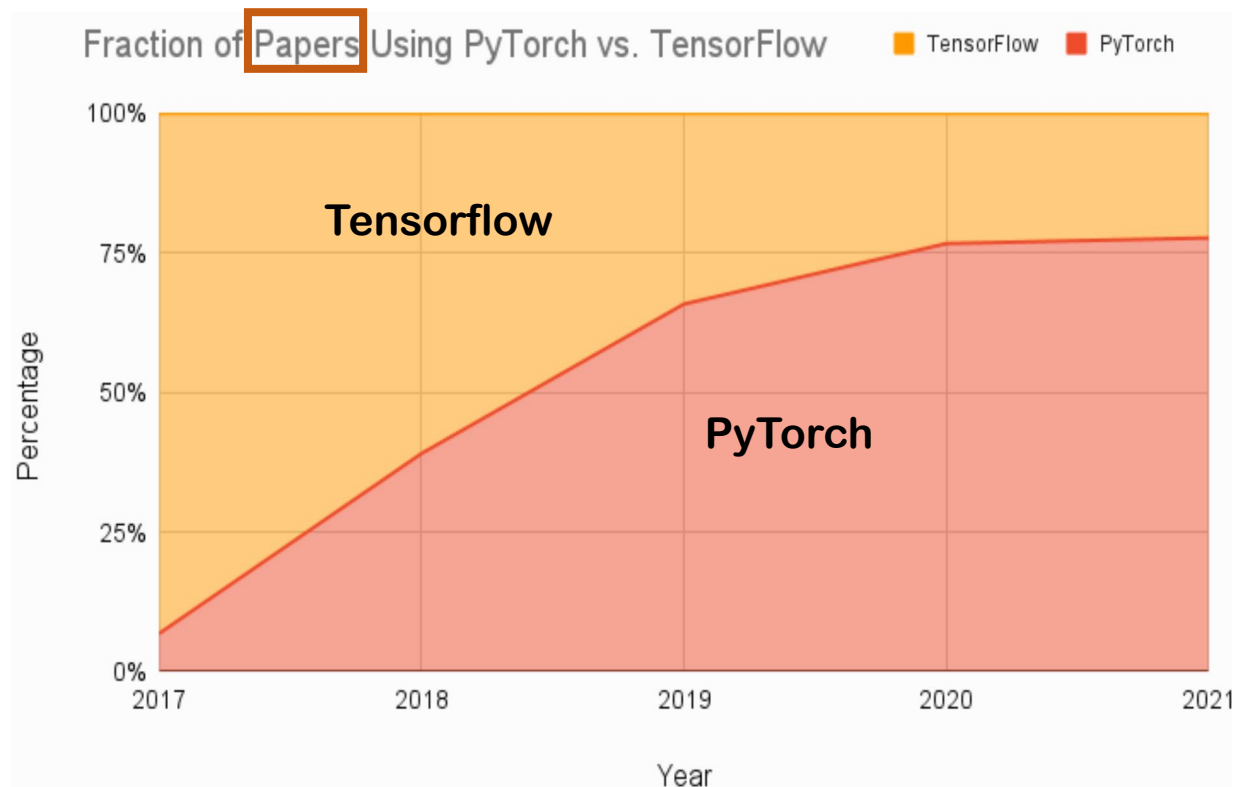
Fall Semester 2026

Adjunct Professor: Donghoon Kang



PyTorch Basics

PyTorch vs. TensorFlow



Creating Tensor

PyTorch

A PyTorch **tensor** is a multi-dimensional **array** that is optimized for numerical computation.

```
import torch
```

```
# Create tensors
```

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])
```

```
print(a)                # tensor([1, 2, 3])
print(b)                # 2 rows, 3 cols
```

```
print(a.ndim)           # number of axes: 1
print(b.ndim)           # number of axes: 2
print(b.shape)          # torch.Size([2, 3]) -> 2 rows, 3 columns
print(b.dtype)          # data type (e.g., torch.int64)
```

```
# Make tensors of zeros/ones or a range
```

```
z = torch.zeros((2, 3))          # shape 2x3, all zeros (float)
o = torch.ones((3, 2), dtype=torch.int32) # shape 3x2, all ones (int)
r = torch.arange(0, 10, 2)        # [0, 2, 4, 6, 8]
lin = torch.linspace(0, 1, steps=5) # [0., 0.25, 0.5, 0.75, 1.]
print(z, o, r, lin, sep="\n")
```

```
tensor([1, 2, 3])
tensor([[1, 2, 3],
        [4, 5, 6]])
1
2
torch.Size([2, 3])
torch.int64
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[1, 1],
        [1, 1]], dtype=torch.int32)
tensor([0, 2, 4, 6, 8])
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

c02_pytorch01.py

1. Similar to NumPy arrays – they support **efficient numerical operations**.
2. **Automatic differentiation** – when used with `requires_grad=True`, tensors can **track computations for backpropagation in deep learning**.
3. **GPU acceleration** – tensors can be **easily moved to GPUs** (`.to("cuda")`) for faster computation.
4. Versatility – they can represent almost any type of data: text embeddings, image pixels, audio signals, etc.

Indexing & Slicing

PyTorch

```
import torch

A = torch.tensor([[10, 11, 12],
                  [20, 21, 22],
                  [30, 31, 32]])

print(A[0, 1].item()) # row 0, col 1 -> 11
print(A[1])           # entire row 1 -> tensor([20, 21, 22])
print(A[:, 2])         # entire col 2 -> tensor([12, 22, 32])
print(A[0:2, 1:3])     # rows 0..1, cols 1..2
                      # -> tensor([[11, 12], [21, 22]])

# Boolean indexing (filtering)
mask = (A % 20 == 0)   # True where divisible by 20
print(mask)
print(A[mask])         # tensor([20])
```

c02_pytorch02.py

```
tensor([[False, False, False],
        [ True, False, False],
        [False, False, False]])
tensor([20])
```

Numpy

```
import numpy as np

A = np.array([[10, 11, 12],
              [20, 21, 22],
              [30, 31, 32]])

print(A[0, 1])        # row 0, col 1 -> 11
print(A[1])           # entire row 1 -> [20 21 22]
print(A[:, 2])         # entire col 2 -> [12 22 32]
print(A[0:2, 1:3])     # rows 0..1, cols 1..2
                      # -> [[11 12], [21 22]]

# Boolean indexing (filtering)
mask = (A % 20 == 0)   # True where divisible by 20
print(mask)
print(A[mask])         # [20]
```

```
[[False False False]
 [ True False False]
 [False False False]]
[20]
```

Examples: shape of PyTorch

```
import torch

a = torch.tensor([])
print(a.shape)

b = torch.tensor([1, 2, 3])
print(b.shape)
print(b.unsqueeze(0).shape)
print(b.unsqueeze(1).shape)

c = torch.tensor([[]])
print(c.shape)

d = torch.tensor([[1, 2, 3]])
print(d.shape)

e = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(e.shape)

f = torch.tensor([[1, 2], [3, 4], [5, 6]])
print(f.shape)

g = torch.tensor([[[1], [2], [3]], [[4], [5], [6]]])
print(g.shape)
```



Examples: shape of PyTorch

```
import torch

a = torch.tensor([])
print(a.shape)

b = torch.tensor([1, 2, 3])
print(b.shape)
print(b.unsqueeze(0).shape)
print(b.unsqueeze(1).shape)

c = torch.tensor([[]])
print(c.shape)

d = torch.tensor([[1, 2, 3]])
print(d.shape)

e = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(e.shape)

f = torch.tensor([[1, 2], [3, 4], [5, 6]])
print(f.shape)

g = torch.tensor([[[1], [2], [3]], [[4], [5], [6]]])
print(g.shape)
```

```
torch.Size([0])
torch.Size([3])
torch.Size([1, 3])
torch.Size([3, 1])
torch.Size([1, 0])
torch.Size([1, 3])
torch.Size([2, 3])
torch.Size([3, 2])
torch.Size([2, 3, 1])
```

Vectorization

performing operations on entire arrays (vectors, matrices, tensors) at once, instead of using explicit Python for loops.

PyTorch

```
import torch

a = torch.arange(5) # tensor([0, 1, 2, 3, 4])
b = torch.arange(5) # tensor([0, 1, 2, 3, 4])

result = a + b # vectorization
print(result) # tensor([0, 2, 4, 6, 8])

numbers = torch.arange(5) # tensor([0, 1, 2, 3, 4])
squared = numbers ** 2 # vectorization
print(squared) # tensor([0, 1, 4, 9, 16])
```

c02_pytorch03.py

NumPy

```
import numpy as np

a = np.arange(5)
b = np.arange(5)

result = a + b # vectorization
print(result) # [0 2 4 6 8]

numbers = np.arange(5)
squared = numbers ** 2 # vectorization
print(squared) # [0 1 4 9 16]
```

list, for loop

```
a = list(range(5))
b = list(range(5))

result = [a[i] + b[i] for i in range(len(a))]
# without vectorization
# [0, 2, 4, 6, 8]
```

```
numbers = list(range(5))
squared = [x**2 for x in numbers]
# without vectorization

print(squared) # [0, 1, 4, 9, 16]
```

Broadcasting

(different shapes that “fit”)

PyTorch

```
import torch

A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]]) # shape (2,3)

d = 10
print(A + d) # tensor([[11, 12, 13],
#                  [14, 15, 16]])

print(type(d)) # <class 'int'>
print(type(A + d)) # <class 'torch.Tensor'>
```

```
import torch

A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]]) # shape (2,3)
b = torch.tensor([10, 20, 30]) # shape (3,)

print(A + b)
# tensor([[11, 22, 33],
#         [14, 25, 36]])
```

NumPy

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)

d = 10
print(A + d) # [[11 22 33]
#             [14 25 36]]

print(type(d)) # <class 'int'>
print(type(A + d)) # <class 'numpy.ndarray'>
```

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)
b = np.array([10, 20, 30]) # shape (3,)

print(A + b)
# [[11 22 33]
#   [14 25 36]]
```

```

import torch

B = torch.tensor([[1, 2, 3],
                  [4, 5, 6]], dtype=torch.float32)
# set dtype=float for convenient mean calculation

print(B.sum())           # tensor(21.)
print(B.mean(), B.min(), B.max())
# tensor(3.5) tensor(1.) tensor(6.)

print(B.median(), B.std()) # tensor(3.5), tensor(1.8708)

# dim in PyTorch -----
print(B.sum(dim=0))      # column sums -> tensor([5., 7., 9.])
print(B.sum(dim=1))      # row sums    -> tensor([ 6., 15.])

# column-wise mean (dim=0)
col_means = B.mean(dim=0)
print("axis=0 (columns) mean:", col_means)
# tensor([2.5000, 3.5000, 4.5000])
axis=0 (columns) mean: tensor([2.5000, 3.5000, 4.5000])

# row-wise mean (dim=1)
row_means = B.mean(dim=1)
print("axis=1 (rows) mean:", row_means) # tensor([2., 5.])

```

```

import numpy as np

B = np.array([[1, 2, 3],
              [4, 5, 6]])

print(B.sum())           # 21
print(B.mean(), B.min(), B.max()) # 3.5 1 6
print(np.median(B), np.std(B))    # 3.5, 1.7078

# axis -----
print(B.sum(axis=0))      # column sums -> [5 7 9]
print(B.sum(axis=1))      # row sums    -> [ 6 15]

# column-wise mean (axis=0)
col_means = B.mean(axis=0)
print("axis=0 (columns) mean:", col_means)
# [2.5 3.5 4.5]
axis=0 (columns) mean: [2.5 3.5 4.5]

# row-wise mean (axis=1)
row_means = B.mean(axis=1)
print("axis=1 (rows) mean:", row_means) # [2. 5.]

```

torch.std() vs np.std()

PyTorch

```
import torch

A = torch.tensor([1, 2, 3, 4, 5, 6], dtype = float)

print(A.median(), A.std())           # tensor(3.), tensor(1.8708)
print(torch.median(A), torch.std(A)) # the same as the above
print(torch.std(A, unbiased=False))  # matches np.std
```

```
tensor(3., dtype=torch.float64) tensor(1.8708, dtype=torch.float64)
tensor(3., dtype=torch.float64) tensor(1.8708, dtype=torch.float64)
tensor(1.7078, dtype=torch.float64)
```

PyTorch computes the **sample** standard deviation:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$
$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)^2}$$
$$s = \sqrt{\frac{1}{5} (2 * (2.5^2 + 1.5^2 + 0.5^2))}$$

Key Difference

The key difference lies in the denominator:

- NumPy: population(N)
- PyTorch (default): sample ($N - 1$)

NumPy

```
import numpy as np

B = np.array([1, 2, 3, 4, 5, 6])

print(np.median(B), np.std(B)) # 3.5, 1.7078
```

3.5 1.707825127659933

NumPy computes the **population** standard deviation:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$
$$\sigma = \sqrt{\frac{1}{6} (2 * (2.5^2 + 1.5^2 + 0.5^2))}$$

reshape(), view()

PyTorch

```
import torch

C = torch.arange(12)      # tensor([0, 1, ..., 11])
print(C.reshape(3, 4))   # shape (3,4)
print(C.reshape(-1, 6))  # -1 lets PyTorch infer -> (2,6)
print(C.view(-1))        # flat view if possible
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
tensor([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11]])
```

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

c02_pytorch06.py

NumPy

```
import numpy as np

C = np.arange(12)        # [0..11]
print(C.reshape(3, 4))  # shape (3,4)
print(C.reshape(-1, 6)) # -1 lets NumPy infer -> (2,6)
print(C.ravel())         # flat view if possible
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

reshape(), view()

PyTorch

```
# Create a contiguous tensor
```

```
A = torch.arange(12)
print("A:", A)
```

```
# Reshape works fine
```

```
print("A.reshape(3,4):\n", A.reshape(3,4))
```

```
# Make a non-contiguous tensor by transposing
```

```
B = A.reshape(3,4).t() # transpose
```

```
C = A.reshape(3,4)
```

```
print("\nB (transposed):\n", B)
```

```
print("Is B contiguous?", B.is_contiguous())
```

```
print("Is C contiguous?", C.is_contiguous())
```

```
tensor([[ 0,  4,  8],
        [ 1,  5,  9],
        [ 2,  6, 10],
        [ 3,  7, 11]])
```

```
Is B contiguous? False
```

```
Is C contiguous? True
```

```
# view requires contiguous memory -> will raise an error
```

```
try:
```

```
    print(B.view(12)) # RuntimeError
```

```
except RuntimeError as e:
```

```
    print("B.view(12) failed:", e)
```

```
B.view(12) failed: view size is not compatible with input tensor's size and stride (at least one dimension spans across two contiguous subspaces). Use .reshape(...) instead.
```

```
# reshape can handle non-contiguous tensors by copying
```

```
print("B.reshape(12):", B.reshape(12)) # works
```

```
B.reshape(12): tensor([ 0,  4,  8,  1,  5,  9,  2,  6, 10,  3,  7, 11])
```

- **view** → requires **contiguous** memory, otherwise it raises an error
- **reshape** → automatically creates a copy if needed and returns the result

reshape().t() vs flatten()

PyTorch

```
E = torch.arange(12).reshape(3,4).t() # non-contiguous
print("Is contiguous?", E.is_contiguous())
```

```
# flatten → always returns contiguous copy
print(E.flatten().is_contiguous()) # True
```

```
# view(-1) → only works if contiguous
try:
```

```
    print(E.view(-1))
except RuntimeError as e:
    print("view failed:", e)
```

```
Is contiguous? False
True
```

c02_pytorch06.py

```
view failed: view size is not compatible with input tensor's size and stride (at least one dimension spans a
cross two contiguous subspaces). Use .reshape(...) instead.
```

view()

```
import torch
```

```
t = torch.arange(12).view(3, 4)
print("reshaped (3x4):\n", t)
```

```
reshaped (3x4):
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

`B = A.reshape(3,4).t()` -> non-contiguous order in memory

Original contiguous buffer



0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

`reshape(3,4)` just change the view of memory: –only the *view* (shape/stride) changes, memory stays the same.

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

`B = A.reshape(3,4).t()` -> looks like a 4×3 matrix, but the elements are accessed in a non-contiguous order.

0	4	8	1	5	9	2	6	10	3	7	11
---	---	---	---	---	---	---	---	----	---	---	----

PyTorch

view

- Only if contiguous
- Fast (just metadata change)
- Raises error otherwise

reshape

- If the data is stored in a simple, contiguous block (e.g., created with `arange`, `zeros`, etc.),
→ reshape just changes the shape.
→ This is a view (very fast, no new memory).
- If the data is not contiguous (e.g., after transpose, slicing with steps, etc.),
→ reshape cannot simply reinterpret memory.
→ It will make a copy internally to give you the desired shape.

`torch.tensor([1., 2.])` is a 1-D tensor (=rank 1) whose shape is (2,).

- In both NumPy and PyTorch, a 1-D vector has no explicit row or column orientation.
- Depending on the operation, PyTorch (and NumPy) implicitly treats it as a row or as a column.

```
import torch

A = torch.tensor([[1., 2.],
                  [3., 4.]])

b = torch.tensor([1., 2.]) # define vector b

row = b[None, :]          # shape: (1, 2) ← row vector
col = b[:, None]          # shape: (2, 1) ← column vector

# Equivalent forms
row2 = b.reshape(1, -1)   # (1, 2)
col2 = b.reshape(-1, 1)   # (2, 1)

# Matrix multiplication
print(b @ A)              # (2,) @ (2,2) → (2,)   (behaves like a row in this context)
print(A @ b)              # (2,2) @ (2,) → (2,)   (behaves like a column in this context)

print(row @ A)            # (1,2) @ (2,2) → (1,2)
print(A @ col)            # (2,2) @ (2,1) → (2,1)
```

`row = b[None, :]`

`None` tells PyTorch to ‘add a new axis (dimension).’

```
tensor([ 7., 10.])
tensor([ 5., 11.])
tensor([[ 7., 10.]])
tensor([[ 5.],
        [11.]])
```



torch.stack()

torch.stack() → adds a new axis.

```
import torch

X = torch.tensor([1, 2, 3])
Y = torch.tensor([4, 5, 6])

print(torch.stack([X, Y], dim=0)) # [[1, 2, 3], tensor([[1, 2, 3],
# [4, 5, 6]] [4, 5, 6]])

print(torch.stack([X, Y], dim=1)) # [[1, 4], tensor([[1, 4],
# [2, 5], [2, 5],
# [3, 6]] [3, 6]])

print(torch.vstack([X, Y])) # [[1, 2, 3], tensor([[1, 2, 3],
# [4, 5, 6]] [4, 5, 6]])

print(torch.hstack([X, Y])) # [1, 2, 3, 4, 5, 6]
# tensor([1, 2, 3, 4, 5, 6])
```

```
import torch

X = torch.tensor([1, 2, 3])
Y = torch.tensor([4, 5, 6])
print(X.shape, Y.shape)

a = torch.stack([X, Y], dim=0)
b = torch.stack([X, Y], dim=1)

print(a.shape) # torch.Size([2, 3])
print(b.shape) # torch.Size([3, 2])

torch.Size([3]) torch.Size([3])
torch.Size([2, 3])
torch.Size([3, 2])
```

c02_pytorch08.py

Linear algebra

```
import torch
```

c02_pytorch09.py

```
A = torch.tensor([[2., 1.],  
                  [1., 3.]])
```

```
b = torch.tensor([1., 2.])
```

```
# Solve  $Ax = b$ 
```

```
x = torch.linalg.solve(A, b)
```

```
print("solution:", x)
```

```
solution: tensor([0.2000, 0.6000])
```

```
# Matrix multiply (dot product)
```

```
M = torch.tensor([[1, 2],  
                  [3, 4]], dtype=torch.float64)
```

```
N = torch.tensor([[10, 20],  
                  [30, 40]], dtype=torch.float64)
```

```
print(M * N)           # element-wise multiplication
```

```
print(M @ N)           # matrix multiplication
```

```
print(torch.matmul(M, N)) # matrix multiplication (same as @)
```

```
tensor([[ 10.,  40.],  
        [ 90., 160.]], dtype=torch.float64)
```

```
tensor([[ 70., 100.],  
        [150., 220.]], dtype=torch.float64)
```

```
tensor([[ 70., 100.],  
        [150., 220.]], dtype=torch.float64)
```

```
# Eigenvalues & eigenvectors
```

```
eigvals, eigvecs = torch.linalg.eig(M)
```

```
print("eigenvalues:", eigvals)
```

```
print("eigenvectors:\n", eigvecs)
```

```
eigenvalues: tensor([-0.3723+0.j,  5.3723+0.j], dtype=torch.complex128)
```

```
eigenvectors:
```

```
tensor([[-0.8246+0.j, -0.4160+0.j],  
        [ 0.5658+0.j, -0.9094+0.j]], dtype=torch.complex128)
```

Converting between NumPy and PyTorch Tensor

```
import torch
import numpy as np

np_array = np.array([1, 2, 3])

# Numpy -> Tensor
tensor_from_np = torch.from_numpy(np_array)
print("Tensor from numpy:", tensor_from_np)

# Tensor -> Numpy
back_to_np = tensor_from_np.numpy()
print("Back to numpy:", back_to_np)
```

```
Tensor from numpy: tensor([1, 2, 3])
Back to numpy: [1 2 3]
```

c02_pytorch10.py

Using GPU

```
# device (use GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

a = torch.tensor([1., 2.], device=device)
b = torch.tensor([1., 2.])
print(a)
print(b)
```

tensor([1., 2.], device='cuda:0')
tensor([1., 2.])

c02_pytorch10.py



CPU



GPU

torch.linspace(0, 1, N).unsqueeze(0)

```
import torch
```

```
N = 5
```

```
x = torch.linspace(0, 1, N)
```

```
print(x)
```

```
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

```
y = torch.linspace(0, 1, N).unsqueeze(0)
```

```
print(y)
```

```
tensor([[0.0000, 0.2500, 0.5000, 0.7500, 1.0000]])
```

```
z = torch.linspace(0, 1, N).unsqueeze(1)
```

```
print(z)
```

```
tensor([[0.0000],  
        [0.2500],  
        [0.5000],  
        [0.7500],  
        [1.0000]])
```

```
print(x.shape, y.shape, z.shape)
```

```
torch.Size([5]) torch.Size([1, 5]) torch.Size([5, 1])
```

Optimizers in `torch.optim`



Optim.SGD

- The most **basic** Stochastic Gradient Descent.
- Supports options like momentum and nesterov

Optim.RMSProp

- Often used for training RNNs



Optim.Adam

- One of the most **widely used** optimizers.
- Based on **Adaptive Moment** Estimation.
- Combines the benefits of **momentum and RMSProp**.



Optim.AdamW

- A variant of Adam with a proper implementation of **weight decay**.
- Standard choice for **Transformer** models such as BERT.

Function Naming Convention in PyTorch

- The trailing underscore `_` in PyTorch convention means **in-place operation (it modifies the tensor directly)**

Example

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])

# Out-of-place operation (creates a new tensor)
y = x.add(10)          # x is unchanged
print("x after x.add(10):", x)

# In-place operation (modifies the tensor directly)
x.add_(10)             # x is changed in place
print("x after x.add_(10):", x)
```

```
x after x.add(10): tensor([1., 2., 3.])
x after x.add_(10): tensor([11., 12., 13.])
```

- `add()` → returns a new tensor, leaves **x unchanged**.
- `add_()` → **modifies** x in-place.

Simplest Example (1): Linear Regression (1D Loss)



We aim to learn the parameters $a, b \in \mathbb{R}$ of a linear function:

$$y = f(x) = ax + b$$

Training Data

Given input samples $\{x_i\}_{i=1}^N$, we generate targets:

$$y_i = a^*x_i + b^* + \epsilon_i$$

where a^*, b^* are the true parameters and $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ is Gaussian noise.

Model Parameters

We initialize parameters randomly:

$$a \sim \mathcal{N}(0, 1), \quad b \sim \mathcal{N}(0, 1).$$

Prediction

For each input x_i , the prediction is:

$$\hat{y}_i = ax_i + b.$$

Simplest Example (1): Linear Regression (1D Loss)



Loss Function

We minimize the mean squared error (MSE):

$$\mathcal{L}(a, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2.$$

Optimization

Parameters are updated with gradient descent (SGD):

$$a \leftarrow a - \eta \frac{\partial \mathcal{L}}{\partial a}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b},$$

where $\eta > 0$ is the learning rate.

Result

After sufficient iterations, the learned parameters

$$(a, b) \approx (a^*, b^*)$$

converge to the true coefficients of the underlying linear function.

Simplest Example (1A): Linear Regression (1D Loss)

c02_pytorchEx01A.py

```

import torch

torch.manual_seed(0)  # Reproducibility

# 1) Ground-truth (random) + data
a_star = torch.randn(1)  # true slope
b_star = torch.randn(1)  # true y-intercept
N = 200
x = torch.linspace(-5, 5, N).unsqueeze(1) # .unsqueeze(1) adds a new dimension at index 1 : making a column vector
noise_std = 0.5
y = a_star * x + b_star + noise_std * torch.randn(N, 1)

# 2) Parameters to learn (scalars with grad)
a = torch.randn(1, requires_grad=True)
b = torch.randn(1, requires_grad=True)

# 3) Optimizer (Adam tends to be steadier than plain SGD here)
opt = torch.optim.Adam([a, b], lr=0.05)

# 4) Training loop
for _ in range(3000):  # more steps → steadier convergence
    y_hat = a * x + b
    loss = torch.mean((y_hat - y) ** 2)

    opt.zero_grad()  # clear old gradients
    loss.backward()  # compute  $\partial L / \partial a$ ,  $\partial L / \partial b$ 
    opt.step()  # update a, b

print(f"True:      a* = {a_star.item():.4f}, b* = {b_star.item():.4f}")
print(f"Learned: a  = {a.item():.4f}, b  = {b.item():.4f}")
print(f"MSE: {loss.item():.4f}")

```

```

True:      a* = 1.5410, b* = -0.2934
Learned: a  = 1.5372, b  = -0.2266
MSE: 0.2349

```

Simplest Example (1B): Linear Regression (1D Loss)

With nn.Module

c02_pytorchEx01B.py

```
import torch
import torch.nn as nn

torch.manual_seed(0)    # Reproducibility

# 1) Ground-truth (random) + data
a_star, b_star = torch.randn(1), torch.randn(1)
N = 100
x = torch.linspace(-5, 5, N).unsqueeze(1)  # .unsqueeze(1) adds a new dimension at index 1 : making a column vector
y = a_star * x + b_star + 0.5 * torch.randn(N, 1)

# 2) Model, loss, optimizer
model = nn.Linear(1, 1)          # y ≈ ax + b (nn.Linear is a subclass of nn.Module), model is an object.
criterion = nn.MSELoss()         # MSELoss is a class and criterion is the instance of MSELoss class
optimizer = torch.optim.SGD(model.parameters(), lr=0.05) # torch.optim.SGD is a class. Optimizer is an object.

# 3) Train
for _ in range(3000):            # more steps → steadier convergence
    y_hat = model(x)
    loss = criterion(y_hat, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

a, b = model.weight.item(), model.bias.item()
print(f"True: a* = {a_star.item():.4f}, b* = {b_star.item():.4f}")
print(f"Learned: a = {a:.4f}, b = {b:.4f}")
print(f"MSE: {loss.item():.4f}")
```

Let `criterion` be an instance of `nn.MSELoss`:

```
criterion = nn.MSELoss().
```

Callable Object via `__call__`

In Python, when we write

```
loss = criterion( $y_{\text{hat}}$ ,  $y$ ),
```

this does not call the constructor again. Instead, it invokes the special method

```
criterion.__call__( $y_{\text{hat}}$ ,  $y$ ).
```

Delegation to forward

For PyTorch modules, the method `__call__` internally delegates to the `forward` function:

```
criterion.__call__( $y_{\text{hat}}$ ,  $y$ )  $\longrightarrow$  criterion.forward( $y_{\text{hat}}$ ,  $y$ ).
```

Mean Squared Error Computation

Specifically, for `nn.MSELoss`, the forward method computes the mean squared error:

$$\mathcal{L}(y_{\text{hat}}, y) = \frac{1}{N} \sum_{i=1}^N (y_{\text{hat},i} - y_i)^2.$$

Summary

Thus,

```
loss = criterion( $y_{\text{hat}}$ ,  $y$ )
```

is equivalent to evaluating the MSE loss between the prediction y_{hat} and the target y .

IMPORTANT

torch.nn.Linear

```
torch.nn.Linear(  
    in_features: int,  
    out_features: int,  
    bias: bool = True,  
    device=None,  
    dtype=None  
)
```

in_features: int

- **Size** of each **input** sample's **last dimension**.
- Example: if your input tensor has shape (batch, 3), then in_features=3

out_features: int

- **Size** of each **output** sample's **last dimension**.
- Example: if out_features=1, the output shape will be (batch, 1).

bias : bool (default is True)

- If True, the layer learns an additive **bias** term
- If False, no bias is added (the computation is just $y = xW$)

device : torch.device (optional)

- Specifies the device where parameters (weights and bias) will be allocated.
- Example: device="cuda:0" puts the layer on the first GPU.

dtype : torch.dtype (optional)

- Specifies the data type of the parameters.
- Example: dtype=torch.float64 creates parameters in double precision.

torch.nn.Linear

```
import torch
import torch.nn as nn

layer = nn.Linear(3, 2)

# fan_in=3 (number of input neurons)
# fan_out=2 (number of output neurons)

# bias=True (default)
```

```
import torch
import torch.nn as nn

# Basic usage
layer1 = nn.Linear(3, 2)
# fan_in=3, fan_out=2, bias=True (default)

# Without bias
layer2 = nn.Linear(3, 1, bias=False)

# On GPU
layer3 = nn.Linear(5, 4, device="cuda")

# With double precision
layer4 = nn.Linear(10, 2, dtype=torch.float64)
```

PyTorch nn.Linear Examples in Row Convention

1. layer1 = nn.Linear(3, 2) (with bias)

$$x = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \in \mathbb{R}^{1 \times 3}, \quad W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad b = \begin{bmatrix} b_1 & b_2 \end{bmatrix} \in \mathbb{R}^{1 \times 2}$$
$$y = xW + b = \begin{bmatrix} y_1 & y_2 \end{bmatrix} \in \mathbb{R}^{1 \times 2}$$

2. layer2 = nn.Linear(3, 1, bias=False)

$$x = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \in \mathbb{R}^{1 \times 3}, \quad W = \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \end{bmatrix} \in \mathbb{R}^{3 \times 1}$$
$$y = xW = \begin{bmatrix} y_1 \end{bmatrix} \in \mathbb{R}^{1 \times 1}$$

nn.Linear in PyTorch

```
import torch
import torch.nn as nn

layer = nn.Linear(in_features=4, out_features=3)

# y = x @ layer.weight.T + layer.bias
# x: input
# y: output
print(f"layer.weight : {layer.weight}") # weight shape = (3, 4)
print(f"layer.bias   : {layer.bias}") # bias shape = (3,)
```

PyTorch performs the linear transformation as:

$$y = xW^T + b.$$

This ensures that:

$$x \in \mathbb{R}^{(B \times I)}, \quad W^T \in \mathbb{R}^{(I \times O)} \quad \Rightarrow \quad xW^T \in \mathbb{R}^{(B \times O)},$$

which matches the expected output shape.

torch.arange() in PyTorch

```
import torch

a = torch.arange(5)
print(a)

b = torch.arange(2, 10)
print(b)

c = torch.arange(2, 10, 2)
print(c)
```

```
tensor([0, 1, 2, 3, 4])
tensor([2, 3, 4, 5, 6, 7, 8, 9])
tensor([2, 4, 6, 8])
```

`torch.arange(start, end, step, dtype, device)`

- start (optional) — first value (default: 0)
- end — stops before this value
- step (optional) — spacing between values (default 1)
- dtype (optional) — data type (e.g., torch.float32)
- device (optional) — CPU/GPU location (default: CPU)

```
import torch

d = torch.arange(5, device='cuda')
print(d)

e = torch.arange(5, device='cpu')
print(e)
```

```
tensor([0, 1, 2, 3, 4], device='cuda:0')
tensor([0, 1, 2, 3, 4])
```

unsqueeze() in PyTorch

```
import torch

row = torch.arange(3).unsqueeze(0) # shape (1,3)
col = torch.arange(3).unsqueeze(1) # shape (3,1)

print("row: ", row)
print("col: ", col)
```

```
row: tensor([[0, 1, 2]])
col: tensor([[0],
            [1],
            [2]])
```

```
# element-wise multiplication
print("col*row : ", col*row) # broadcast
print("row*col : ", row*col) # broadcast
```

```
col*row : tensor([[0, 0, 0],
                [0, 1, 2],
                [0, 2, 4]])
row*col : tensor([[0, 0, 0],
                [0, 1, 2],
                [0, 2, 4]])
```

torch.arange does not create matrices.

To expand dimension, we must use `.unsqueeze()` (supports `broadcasting`)

```
col --> [[0, 0, 0],
         [1, 1, 1],
         [2, 2, 2]]
```

```
row --> [[0, 1, 2],
         [0, 1, 2],
         [0, 1, 2]]
```

Both become (3,3).

unsqueeze() in PyTorch

```
import torch

# A simple 1D tensor (vector)
x = torch.tensor([1.0, 2.0, 3.0]) # shape: (3,)
print("x * x", x * x)
print("x @ x", x @ x) # torch.dot(x,x)

x * x tensor([1., 4., 9.])
x @ x tensor(14.)

# Two different unsqueeze operations:
row = x.unsqueeze(0) # shape: (1, 3)
col = x.unsqueeze(1) # shape: (3, 1)

# element-wise multiplication (broadcasting is required)
print("col*row : ", col*row) # broadcast to (3,3)*(3,3) = (3,3)
print("row*col : ", row*col) # broadcast to (3,3)*(3,3) = (3,3)

# matrix multiplication
print("col@row : ", col@row) # (3,1) @ (1,3) = (3,3)
print("row@col : ", row@col) # (1,3) @ (3,1) = (1,1)
```

```
col*row : tensor([[0, 0, 0],
                 [0, 1, 2],
                 [0, 2, 4]])
row*col : tensor([[0, 0, 0],
                 [0, 1, 2],
                 [0, 2, 4]])
```

```
col@row : tensor([[0, 0, 0],
                 [0, 1, 2],
                 [0, 2, 4]])
row@col : tensor([[5]])
```

Attributes of torch.nn.Linear

```
import torch
import torch.nn as nn

layer = nn.Linear(3, 2)
# layer = nn.Linear(in_features=3, out_features=2, bias=True)

print(layer.in_features)    # 3
print(layer.out_features)   # 2
print(layer.weight.shape)   # torch.Size([2, 3])
print(layer.bias.shape)     # torch.Size([2])
print(layer.training)
print("state dict keys:", layer.state_dict().keys())
```

```
3
2
torch.Size([2, 3])
torch.Size([2])
```

```
print("training mode:", layer.training)
print("state dict keys:", layer.state_dict().keys())
```

```
training mode: True
state dict keys: odict_keys(['weight', 'bias'])
```

Because **nn.Linear** is a **subclass of nn.Module**, it also has many other attributes, such as:

- `training` → whether the layer is in training or evaluation mode.
- `parameters()` → a method that returns all learnable parameters (weight and bias).
- `state_dict()` → a method that returns all parameters and buffers in a dictionary.
- `to(device)` → moves parameters to a specific device (CPU/GPU).

```
import torch
import torch.nn as nn

layer = nn.Linear(3, 2)
# layer = nn.Linear(in_features=3, out_features=2, bias=True)

print(layer.in_features)    # 3
print(layer.out_features)   # 2
print(layer.weight.shape)   # torch.Size([2, 3])
print(layer.bias.shape)     # torch.Size([2])
print(layer.training)
print("state dict keys:", layer.state_dict().keys())
```

```
3
2
torch.Size([2, 3])
torch.Size([2])
```

```
print("training mode:", layer.training)
print("state dict keys:", layer.state_dict().keys())
```

```
training mode: True
state dict keys: odict_keys(['weight', 'bias'])
```

Because **nn.Linear** is a **subclass of nn.Module**, it also has many other attributes, such as:

- `training` → whether the layer is in training or evaluation mode.
- `parameters()` → a method that returns all learnable parameters (weight and bias).
- `state_dict()` → a method that returns all parameters and buffers in a dictionary.
- `to(device)` → moves parameters to a specific device (CPU/GPU).

register_buffer() in nn.Module

```
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(3))
        self.register_buffer("scale", torch.tensor(2.0))

model = MyModel()
print(model.state_dict())
```

```
OrderedDict([('weight', tensor([-0.9091, -0.1381,  0.6516])), ('scale', tensor(2.))])
```

In PyTorch, `register_buffer(name, tensor)` is a method of `nn.Module` that attaches a tensor to the module without treating it as a learnable parameter.

A buffer is a tensor that:

- belongs to the module,
- is saved and loaded with `state_dict()`,
- moves automatically with `model.to(device)`,
- is **not** updated by the optimizer.

Parameters vs. Buffer in nn.Module

	Stored in model	Updated by optimizer	Moves with <code>to(device)</code>	In <code>state_dict()</code>
Parameter	Yes	Yes	Yes	Yes
Buffer	Yes	No	Yes	Yes

A buffer is ideal for tensors that are part of the model but should not be trained.

Why Use Buffer ?

Some tensors should remain part of the model's state but never receive gradients.

Examples include:

- sinusoidal positional encodings,
- BatchNorm running statistics,
- constant masks,
- fixed lookup tables,
- normalization constants.

Why do we need state_dict() ?

```
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(3))
        self.register_buffer("scale", torch.tensor(2.0))

model = MyModel()
print (model.state_dict())
```

```
model.state_dict()
```

returns a dictionary similar to:

```
{
    'weight': tensor([...]),
    'scale': tensor(2.)
}
```

```
OrderedDict([('weight', tensor([-0.9091, -0.1381,  0.6516])), ('scale', tensor(2.))])
```

In PyTorch, `model.state_dict()` returns a Python dictionary containing the complete internal state of a model. It includes all trainable parameters and all registered buffers. This dictionary represents the essential data required to save or load a model reliably.

What does `state_dict()` contains ?

(1) Trainable Parameters

These are tensors wrapped in `nn.Parameter`, such as:

- weights,
- biases,
- linear layer matrices,
- convolution kernels.

(2) Registered Buffers

Buffers are persistent tensors registered using:

```
self.register_buffer("name", tensor)
```

Examples include:

- sinusoidal positional encodings,
- BatchNorm running mean and variance,
- fixed masks,
- lookup tables or constants.

Save and Load Models using PyTorch

```
import torch
import torch.nn as nn

# -----
# Simple model definition
# -----
class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(4, 3)

    def forward(self, x):
        return self.linear(x)

# -----
# 1. Create a model and inspect initial weights
# -----
model = SimpleModel()
print("Initial model weights:")
print(model.linear.weight)
print()
```

```
Initial model weights:
Parameter containing:
tensor([[ 0.2515, -0.3357,  0.1317,  0.3983],
        [-0.4345,  0.2308, -0.3156,  0.4657],
        [ 0.0736, -0.1300,  0.4589, -0.1545]], requires_grad=True)
```

Save and Load Models using PyTorch

```
# -----  
# 2. Save model weights to a file  
# -----  
torch.save(model.state_dict(), "model.pth")  
print("Model saved to model.pth")  
print()  
  
# -----  
# 3. Create a NEW model instance (untrained / random weights)  
# -----  
new_model = SimpleModel()  
print("New model BEFORE loading:")  
print(new_model.linear.weight)  
print()
```

```
Model saved to model.pth
```

```
New model BEFORE loading:
```

```
Parameter containing:
```

```
tensor([[ -0.1537,  0.1422,  0.1379,  0.1984],  
        [ -0.3955, -0.4262,  0.2577, -0.4344],  
        [ 0.1168,  0.0167,  0.1256,  0.3544]], requires_grad=True)
```

Save and Load Models using PyTorch

```
# -----  
# 4. Load the saved weights into the new model  
# -----  
state_dict = torch.load("model.pth", weights_only=True)  
new_model.load_state_dict(state_dict)  
  
print("New model AFTER loading saved weights:")  
print(new_model.linear.weight)  
print()  
  
# -----  
# 5. Verify the two models now match  
# -----  
print("Do the weights match?", torch.allclose(model.linear.weight, new_model.linear.weight))
```

```
New model AFTER loading saved weights:  
Parameter containing:  
tensor([[ 0.2515, -0.3357,  0.1317,  0.3983],  
        [-0.4345,  0.2308, -0.3156,  0.4657],  
        [ 0.0736, -0.1300,  0.4589, -0.1545]], requires_grad=True)  
  
Do the weights match? True
```

torch.allclose() in PyTorch

```
import torch
import torch.nn as nn

a = torch.tensor([1.0])
b = torch.tensor([1.0 + 1e-7])

print(torch.equal(a, b))      # False
print(torch.allclose(a, b))  # True
```

Common use case: **verifying model loading**

torch.Tensor.masked_fill() in PyTorch

```
import torch

x = torch.tensor([1.0, 2.0, 3.0, 4.0])

# Create a boolean mask
mask = torch.tensor([False, True, False, True])

# Use masked_fill
y = x.masked_fill(mask, -5.0) # elements with True in the mask are replaced by -5.0

print("Original tensor:", x)
print("Mask:", mask)
print("After masked_fill:", y)
```

```
Original tensor: tensor([1., 2., 3., 4.])
Mask: tensor([False,  True, False,  True])
After masked_fill: tensor([ 1., -5.,  3., -5.])
```

Simple Example (2): Linear Regression



Let the input data matrix, weight vector, and target value be

$$X \in \mathbb{R}^{N \times d}, \quad w \in \mathbb{R}^{d \times 1}, \quad t \in \mathbb{R}^{N \times 1},$$

respectively. Here, $N = 5$ and $d = 3$ (5 samples, 3 features).

Predictions without bias:

$$\hat{y} = Xw.$$

Loss Function (MSE: Mean Squared Error)

$$L(w) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - t_i)^2 = \frac{1}{N} \|Xw - t\|_2^2.$$

If we use a scalar target c (broadcasted to all samples), then $t_i = c$ and:

$$L(w) = \frac{1}{N} \sum_{i=1}^N (x_i^\top w - c)^2.$$

target

$$t \in \mathbb{R}^{N \times 1}, \quad t = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

Gradients

$$\nabla_w L(w) = \frac{2}{N} X^\top (Xw - t),$$

Simple Example (2): Linear Regression

Without bias

```
import torch

# device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# inputs
x = torch.randn(5, 3, device=device) # 5 samples, each with 3 features
target = torch.tensor([[0.5]]*5, device=device) # The shape of target vector (all 0.5) is (5,1).

# parameter to learn
w = torch.randn(3, 1, device=device, requires_grad=True) # random values are sampled from N(0,1)
# requires_grad=True tells autograd to track this tensor and compute gradients

# optimizer
optimizer = torch.optim.SGD([w], lr=0.1)

for step in range(20):
    pred = x @ w # The shape of pred is (5,1). @ does matrix multiplication.
    loss = ((pred - target)**2).mean() # MSE loss

    # autograd (= automatically computes gradients of tensors)
    optimizer.zero_grad() # Reset gradients from the previous step (set .grad to zero)
    loss.backward() # Compute gradients of loss (=d(loss)/dw) w.r.t parameters → stored in w.grad
    optimizer.step() # Update w using gradients: w = w - lr * w.grad

    if step % 5 == 0: # print every 5 steps
        print(f"Step {step:2d} | loss {loss.item():.4f} | w = {w.detach().T}")
        # .detach() breaks the tensor away from the autograd graph (so it won't track gradients).

print("Final w:", w.detach().T)
```

```
Step 0 | loss 0.8743 | w = tensor([[ 0.6639, -0.4362,  0.3074]], device='cuda:0')
Step 5 | loss 0.1398 | w = tensor([[ 0.2987, -0.5716,  0.0984]], device='cuda:0')
Step 10 | loss 0.0936 | w = tensor([[ 0.2158, -0.5652,  0.0095]], device='cuda:0')
Step 15 | loss 0.0859 | w = tensor([[ 0.1982, -0.5498, -0.0406]], device='cuda:0')
Final w: tensor([[ 0.1977, -0.5432, -0.0687]], device='cuda:0')
```

Simple Example (2): Linear Regression using torch.nn

```
import torch
import torch.nn as nn
```

```
# device (use GPU if available)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
# inputs
```

```
x = torch.randn(5, 3, device=device)          # 5 samples, 3 features each (shape: 5x3)
target = torch.full((5, 1), 0.5, device=device) # target values: constant 0.5 (shape: 5x1)
```

```
# model:  $y = x @ w$ , no bias (same as the original manual implementation)
model = nn.Linear(3, 1, bias=False).to(device)
```

Without bias

```
# optimizer & loss function
```

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
loss_fn = nn.MSELoss()
```

```
for step in range(20):
```

```
    optimizer.zero_grad(set_to_none=True) # reset gradients
    pred = model(x)                       # forward pass (output shape: 5x1)
    loss = loss_fn(pred, target)          # mean squared error loss
    loss.backward()                       # backpropagation
    optimizer.step()                      # parameter update
```

```
    if step % 5 == 0:
```

```
        with torch.no_grad():
```

```
            # model.weight has shape (1,3), so no need to transpose for printing
```

```
            print(f"Step {step:2d} | loss {loss.item():.4f} | w = {model.weight.detach()}")
```

```
with torch.no_grad():
```

```
    print("Final w:", model.weight.detach())
    print(target.shape)
```

c02_pytorchEx02B.py

with torch.no_grad(): →

temporarily disables gradient tracking when you're only reading values (like model outputs or weights).

Model.weight → a learnable parameter in PyTorch. That means it is a torch.Tensor with `requires_grad=True` so that gradients can be computed during backpropagation.

```
Step 0 | loss 0.4216 | w = tensor([[ -0.4224,  0.5199, -0.5011]], device='cuda:0')
Step 5 | loss 0.2786 | w = tensor([[ -0.2865,  0.5620, -0.3276]], device='cuda:0')
Step 10 | loss 0.2443 | w = tensor([[ -0.2172,  0.5417, -0.2330]], device='cuda:0')
Step 15 | loss 0.2291 | w = tensor([[ -0.1780,  0.5005, -0.1719]], device='cuda:0')
Final w: tensor([[ -0.1569,  0.4636, -0.1352]], device='cuda:0')
torch.Size([5, 1])
```

We use `.detach()` to safely look at the parameter values without gradient tracking.

Adding bias

Let the input data matrix, weight vector, bias, and target value be

$$X \in \mathbb{R}^{N \times d}, \quad w \in \mathbb{R}^{d \times 1}, \quad b \in \mathbb{R}, \quad t \in \mathbb{R}^{N \times 1},$$

respectively. Here, $N = 5$ and $d = 3$ (5 samples, 3 features).

Predictions with bias:

$$\hat{y} = Xw + b\mathbf{1},$$

where $\mathbf{1} \in \mathbb{R}^{N \times 1}$ is a vector of ones.

Loss Function (MSE: Mean Squared Error)

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - t_i)^2 = \frac{1}{N} \|Xw + b\mathbf{1} - t\|_2^2.$$

If we use a scalar target c (broadcasted to all samples), then $t_i = c$ and:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (x_i^\top w + b - c)^2.$$

Simple Example (2): Linear Regression using torch.nn

c02_pytorchEx02B.py

```
import torch
import torch.nn as nn

# device (use GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# inputs
x = torch.randn(5, 3, device=device)          # 5 samples, 3 features each (shape: 5x3)
target = torch.full((5, 1), 0.5, device=device) # target values: constant 0.5 (shape: 5x1)

# model:  $y = x @ w + b$  (bias is added)
model = nn.Linear(3, 1, bias=True).to(device)

# optimizer & loss function
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
loss_fn = nn.MSELoss()

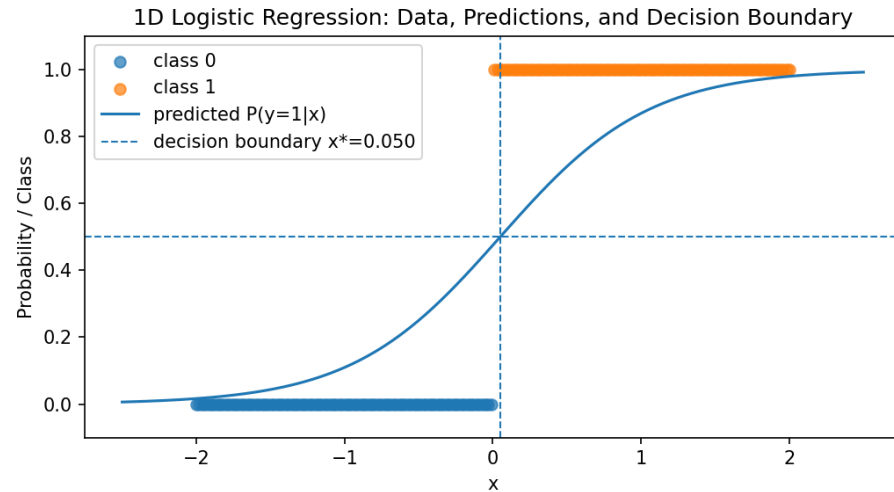
for step in range(20):
    optimizer.zero_grad(set_to_none=True)      # reset gradients
    pred = model(x)                             # forward pass (output shape: 5x1)
    loss = loss_fn(pred, target)                # mean squared error loss
    loss.backward()                             # backpropagation
    optimizer.step()                           # parameter update

    if step % 5 == 0:
        with torch.no_grad():
            # model.weight has shape (1,3), so no need to transpose for printing
            print(f"Step {step:2d} | loss {loss.item():.4f} | w = {model.weight.detach()}")

with torch.no_grad():
    print("Final w:", model.weight.detach())
    print(target.shape)
```

With bias

Simple Example (3): Binary Classification



1. Dataset (1D line)

We construct N (in our case, $N = 200$) evenly spaced points on a 1D line and assign binary labels via a step rule:

$$x_i \in \mathbb{R}, \quad x_i \text{ linspace in } [-2, 2], \quad y_i = \mathbf{1}\{x_i > 0\} \in \{0, 1\}, \quad i = 1, \dots, N.$$

In code: `x = torch.linspace(-2,2,N).unsqueeze(1)`, `y = (x[:,0] > 0).float()`.

2. Model (Logistic Regression)

A single linear (affine) function maps input $x \in \mathbb{R}$ to a scalar:

$$z_i = wx_i + b, \quad w, b \in \mathbb{R}.$$

The sigmoid (logistic) function gives the class probability

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{p}_i = \Pr(y_i = 1 \mid x_i) = \sigma(z_i) = \sigma(wx_i + b).$$

Simple Example (3): Binary Classification

3. Loss: Binary Cross-Entropy (with logits)

For a label $y_i \in \{0, 1\}$ and logit z_i ,

$$\ell(z_i, y_i) = -\left(y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))\right).$$

PyTorch's `BCEWithLogitsLoss` implements a numerically stable form. The empirical risk minimized during training is the mean loss:

$$\mathcal{L}(\boxed{w, b}) = \frac{1}{N} \sum_{i=1}^N \ell(wx_i + b, y_i).$$

4. Gradients (1D closed form)

Using $\frac{d}{dz} \ell(z, y) = \sigma(z) - y$, we get

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{N} \sum_{i=1}^N (\sigma(wx_i + b) - y_i) x_i, \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N} \sum_{i=1}^N (\sigma(wx_i + b) - y_i).$$

Simple Example (3): Binary Classification

```
import torch

# Toy dataset (1D line) Label = 1 if x > 0, else 0
N = 200
x = torch.linspace(-2, 2, N).unsqueeze(1)          # shape: (N,1)
y = (x[:,0] > 0).float()                          # step function

# Logistic regression model
model = torch.nn.Linear(1, 1)                     # w*x + b
criterion = torch.nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

# Training
for step in range(100):
    logits = model(x).squeeze(1)                  # raw scores
    loss = criterion(logits, y)                   # binary CE loss

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if step % 20 == 0:
        with torch.no_grad():
            preds = (logits > 0).float()           # threshold at 0
            acc = (preds == y).float().mean().item()
            print(f"step={step:3d}  loss={loss.item():.4f}  acc={acc:.3f}")
```

step= 0	loss=0.8361	acc=0.500
step= 20	loss=0.4480	acc=0.805
step= 40	loss=0.3181	acc=0.920
step= 60	loss=0.2595	acc=0.955
step= 80	loss=0.2259	acc=0.970

Thank you!

